



Python Programming - 2301CS404

Lab - 12

Roll No : 310

Name : Ronit Bhadania

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

```
In [1]: a = 10
b = 0

try:
    result = a / b
    print(f"Result: {result}")
except ZeroDivisionError:
    print("Error: You cannot divide by zero!")

try:
    n = int("Hello World")
except ValueError:
    print("Error: That string cannot be converted to a number.")

try:
    total = "Total: " + 100
except TypeError:
    print("Error: You cannot mix strings and numbers in this operation.")
except :
    print("Error occured while trying to handle the error...")
```

Error: You cannot divide by zero!

Error: That string cannot be converted to a number.

Error: You cannot mix strings and numbers in this operation.

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

```
In [4]: try:
        my_list = [10, 20]
        item = my_list[5]
    except IndexError:
        print("Error: That list index is out of range.")
    try:
        my_dict = {"name": "Ronit"}
        print(my_dict["age"])
    except KeyError:
        print("Error: That key was not found in the dictionary.")
    except :
        print("Error occured while trying to handle the error...")
```

Error: That list index is out of range.

Error: That key was not found in the dictionary.

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

```
In [5]: try:
        with open("abd.txt", "r") as fp:
            f = fp.read()
            print(f)
    except FileNotFoundError:
        print("Error: The file you're trying to access doesn't exist.")
```

Error: The file you're trying to access doesn't exist.

04) WAP that handles all type of exceptions in a single except block in standard error message format.

```
In [10]: try:
        result = "10" + 5
    except Exception as e:
        print(type(e))
        print(e)
```

<class 'TypeError'>

can only concatenate str (not "int") to str

05) WAP to demonstrate else and finally block.

```
In [22]: balance = 5000
        debit = int(input("Enter amount to debit:"))
        try:
```

```

    if(debit>balance or balance<0) :
        raise ValueError("Debit is greater than balance")
except ValueError as err:
    print(err)
else:
    balance = balance - debit
    print(f"Remaining balance is = {balance}")
finally:
    print("Transaction Completed...")

```

Remaining balance is = 4000

Transaction Completed...

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

```

In [23]: a = 10
        b = 0
        def divide(a, b):
            try:
                result = a / b
                return result
            except ZeroDivisionError:
                return "Error: Division by zero is not allowed."
        print(divide(a,b))

```

Error: Division by zero is not allowed.

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

```

In [34]: def calculate_total() :
        try:
            n = int(input("Enter no. of items:"))
            total=0

            for i in range(n):
                price = int(input(f"Enter price of item {i+1}:"))

                if(price<0):
                    raise ValueError("Price can't be negative!")
                total+=price
            print("Total Bill amount:",total)
        except ValueError as e:
            print(e)
        calculate_total()

```

Total Bill amount: 101

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

```
In [38]: try:
    grade = input("Enter the grades with coma seprate").split(',')

    int_grade = [int(x) for x in grade if int(x)>0 and int(x)<=100]

    print(int_grade)

except Exception as err:
    print(err)
finally:
    print("Execution completed")
```

[10, 20, 30]

Execution completed

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

```
In [39]: try:
    l1 = input("enter 5 marks comma separeted").split(',')
    for i in l1:
        assert int(i)>100, 'Marks should be between 0 and 100.'
except AssertionError as err:
    print(err)
else :
    print("Valid marks entered")
```

error here

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the message "Correct" otherwise.

```
In [43]: class WeakPasswordError(Exception):
    pass
password = input("Enter your password: ")
try:
    if len(password) < 8:
        raise WeakPasswordError("Password must be at least 8 characters long.")
    if not any(char.isdigit() for char in password):
```

```

        raise WeakPasswordError("Password must contain at least one digit.")
    print("Correct")
except WeakPasswordError as e:
    print("WeakPasswordError:", e)

```

WeakPasswordError: Password must be at least 8 characters long.

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

```

In [4]: import math
class NegativeNumberError(Exception) :
    "Cannot calculate te square root of a negative number"
    pass

try :
    a = int(input("Enter the Number"))
    if(a>0):
        print(math.sqrt(a))
    else :
        raise NegativeNumberError("Number is negative")
except NegativeNumberError as e :
    print(e)

```

5.0